

TP n°3 – Équations différentielles

Dans tout le TP, on utilisera les bibliothèques suivantes. Les questions marquées **PAPIER** se traitent sur feuille, celles marquées **PYTHON** sur machine.

TP3.py ×

```
1 import numpy as np
2 from math import *
3 import matplotlib.pyplot as plt
4 from scipy.integrate import solve_ivp
5 from scipy.optimize import fsolve
```

Exercice 1 – Autour des méthodes d'Euler

On se propose d'étudier le problème de Cauchy suivant

$$\begin{cases} y' = \cos(y) + t \\ y(0) = 1 \end{cases}$$

PAPIER Question 1 – Justifier que ce problème admet une unique solution définie sur $\mathbb{R}$. Montrer que cette solution est croissante sur $[1; +\infty[$.

PYTHON Question 2 – Écrire l'équation différentielle sous la forme $y' = f(t, y)$ et implémenter f en Python.

PYTHON Question 3 – Implémenter en Python une fonction `euler_explicite(f, y0, T, n)` qui retourne les listes `t` et `y` obtenues par le schéma d'Euler explicite sur $[0, T]$ avec n subdivisions.

PYTHON Question 4 – Tracer la solution approchée sur $[0, 5]$ pour $N = 50$, $N = 200$ et $N = 1000$ sur un même graphique. On ajoutera des légendes.

PYTHON PAPIER Question 5 – À l'aide de la fonction `scipy.integrate.solve_ivp`, tracer sur un même graphique la solution de référence et les solutions d'Euler explicite pour $n = 50$, $n = 200$ et $n = 1000$ sur $[0, 5]$. On soignera la présentation : couleurs distinctes pour chaque courbe, style pointillé pour les approximations, trait plein pour la solution de référence, et légendes explicites. Que constatez-vous lorsque n augmente ?
Documentation `scipy.integrate.solve_ivp` : https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html

PAPIER Question 6 – Quel est l'ordre de convergence de la méthode d'Euler explicite ?

PAPIER Question 7 – On note $E(n) = \max_k |y_k - y_{\text{ref}}(t_k)|$. En déduire qu'il existe une constante $\tilde{C} > 0$ telle que :

$$\log(E(n)) \leq \tilde{C} + \log(h)$$

PYTHON PAPIER Question 8 – Calculer l'erreur globale $E(n) = \max_k |y_k - y_{\text{ref}}(t_k)|$ pour $n \in \{50, 100, 200, 500, 1000\}$ et tracer $\log(E(n))$ en fonction de $\log(h)$ où $h = T/n$. On fera apparaître la première bissectrice en pointillés. Que constatez-vous ? Quelle est la pente de la droite obtenue ?

PYTHON Question 9 – Implémenter une fonction `euler_implicit(f, y0, T, n)` qui retourne les listes `t` et `y` obtenues par le schéma d'Euler implicite.

On utilisera `scipy.optimize.fsolve` pour résoudre l'équation non linéaire à chaque pas. Tracer sur un même graphique la solution de référence, la solution d'Euler explicite et la solution d'Euler implicite pour $n = 50$.
Documentation `scipy.optimize.fsolve` : <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.fsolve.html>

Exercice 2 – Méthodes explicites d'ordre supérieur

On se propose d'étudier le problème de Cauchy suivant

$$\begin{cases} y' = -y \\ y(0) = 1 \end{cases}$$

dont la solution exacte est $y(t) = e^{-t}$.

PYTHON Question 10 – Écrire l'équation différentielle sous la forme $y' = g(t, y)$ et implémenter g en Python.

PYTHON Question 11 – Implémenter la solution exacte de ce problème de Cauchy.

PYTHON Question 12 – Implémenter en Python une fonction `heun(f, y0, T, n)` qui retourne les listes t et y obtenues par la méthode de Heun sur $[0, T]$ avec n subdivisions.

PYTHON Question 13 – Implémenter en Python une fonction `milieu(f, y0, T, n)` qui retourne les listes t et y obtenues par la méthode du point milieu sur $[0, T]$ avec n subdivisions.

PYTHON Question 14 – Implémenter en Python une fonction `RK4(f, y0, T, n)` qui retourne les listes t et y obtenues par la méthode de Runge-Kutta 4 sur $[0, T]$ avec n subdivisions.

PAPIER Question 15 – Quel est l'ordre de convergence des méthodes de Heun, du point milieu et de Runge-Kutta 4 ?

PYTHON PAPIER Question 16 – Vérifier numériquement en traçant $\log(E(n))$ en fonction de $\log(h)$ pour chacune des méthodes et en faisant apparaître les droites $y = 2x$ et $y = 4x$ en pointillés. Que constatez-vous ?

Exercice 3 – Système différentiel : modèle proie-prédateur de Lotka-Volterra

Le modèle de proie-prédateur permet de décrire la dynamique de populations d'un système biologique où interagissent un prédateur et sa proie. Il a été proposé par Alfred James Lotka et Vito Volterra.

On note $x(t)$ et $y(t)$ les effectifs respectifs des proies et des prédateurs à l'instant t , le modèle s'écrit sous la forme du système d'équations différentielles suivant :

$$\begin{cases} \frac{dx(t)}{dt} = x(t) (\alpha - \beta y(t)) \\ \frac{dy(t)}{dt} = y(t) (\delta x(t) - \gamma) \end{cases}$$

où :

- α est le taux de reproduction intrinsèque des proies ;
- β est le taux de mortalité des proies due aux prédateurs ;
- δ est le taux de reproduction des prédateurs en fonction des proies mangées ;
- γ est le taux de mortalité intrinsèque des prédateurs.

PAPIER Question 17 – Comment interpréter la première équation lorsque $\beta = 0$? Et la seconde lorsque $\delta = 0$? Comment évoluent les populations concernées ?

PAPIER Question 18 – Écrire ce système différentiel sous la forme $Y' = F(t, Y)$ avec $Y = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$. On admet que F vérifie le théorème de Cauchy-Lipschitz.

PYTHON Question 19 – Coder en Python la fonction F telle que $Y' = F(t, Y)$ avec $Y = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$. On prendra $\alpha = \frac{2}{3}$, $\beta = \frac{4}{3}$, $\delta = \gamma = 1$.

PYTHON Question 20 – En déduire une fonction `lotkaVolterra(x0, y0, T, n)` qui prend en argument les effectifs initiaux x_0 et y_0 des proies et prédateurs, un temps T et un entier n , et qui renvoie la liste des instants $(t_k)_{0 \leq k \leq n}$ ainsi que les listes des approximations $(x_k)_{0 \leq k \leq n}$ et $(y_k)_{0 \leq k \leq n}$.

PYTHON PAPIER Question 21 – Représenter graphiquement les effectifs $x(t)$ et $y(t)$ des populations de proies et de prédateurs pour $t \in [0, 50]$ et $(x_0, y_0) = (1, 1)$. Que constatez-vous ?